explicit call to `load()`. As you might expect, `store()` is a store operation, whereas `load()` is a load operation. `exchange()` is a read-modify-write operation:

```
std::atomic<bool> b;
bool x=b.load(std::memory_order_acquire);
b.store(true);
x=b.exchange(false,std::memory_order_acq_rel);
```

`exchange()` isn't the only read-modify-write operation supported by `std::atomic<bool>`; it also introduces an operation to store a new value if the current value is equal to an expected value.

Storing a New Value (or not) Depending on the Current Value

This new operation is called compare/exchange, and it comes in the form of the `compare_exchange_weak()` and `compare_exchange_strong()` member functions. The compare/exchange operation is the cornerstone of programming with atomic types; it compares the value of the atomic variable with a supplied expected value and stores the supplied desired value if they're equal. If the values aren't equal, the expected value is updated with the actual value of the atomic variable. The return type of the compare/exchange functions is a `bool`, which is `true` if the store was performed and `false` otherwise.

For `compare_exchange_weak()`, the store might not be successful even if the original value was equal to the expected value, in which case the value of the variable is unchanged and the return value of `compare_exchange_weak()` is `false`. This is most likely to happen on machines that lack a single compare-and-exchange instruction, if the processor can't guarantee that the operation has been done atomically—possibly because the thread performing the operation was switched out in the middle of the necessary sequence of instructions and another thread scheduled in its place by the operating system where there are more threads than processors. This is called a *spurious failure*, because the reason for the failure is a function of timing rather than the values of the variables.

Because `compare_exchange_weak()` can fail spuriously, it must typically be used in a loop:

```
bool expected=false;
extern atomic<bool> b; // set somewhere else
while(!b.compare_exchange_weak(expected,true) && !expected);
```

In this case, you keep looping as long as `expected` is still `false`, indicating that the `compare_exchange_weak()` call failed spuriously.

On the other hand, `compare_exchange_strong()` is guaranteed to return `false` only if the actual value wasn't equal to the `expected` value. This can eliminate the need for loops like the one shown where you just want to know whether you successfully changed a variable or whether another thread got there first.